pyWSciML

A Python Library for Weak-Form Scientific Machine Learning

Scotty Mcdonough

May 2026

1 Summary

This report documents the design, development, and structure of pyWSciML, a Python library that implements the Weak-form Estimation of Nonlinear Dynamics (WENDy) framework. The library was created by refactoring the original research codebase that accompanied the paper "Weak Form Scientific Machine Learning: Test Function Construction for System Identification" (Tran & Bortz, 2025). That codebase, while scientifically functional, consisted of loosely organized scripts and functions that were difficult for other outside researchers to understand, extend, or reproduce. The goal of this project was to restructure the code into a clean, object-oriented library that implements the algorithms described in the paper while making them accessible to future users and contributors.

2 Background

2.1 The Weak-Form Scientific Machine Learning Framework

Weak form Scientific Machine Learning (WSciML) is a recently developed framework for data-driven modeling and scientific discovery. It addresses a fundamental challenge in system identification: given noisy time-series observations of a dynamical system, how can one reliably recover the governing equations or their parameters?
In the weak-form approach, the governing equation is multiplied by compactly supported test functions and integrated by parts. This transfers the derivative onto the smooth, analytically known test function, yielding an algebraic system that can be solved via least squares. When the model is linear in its parameters, as in the standard WENDy setting, this produces a standard linear regression problem of the form $G \cdot w \approx b$, where G is a matrix constructed from the test-function-weighted library terms, b comes from the test-function-weighted derivative (shifted via integration by parts), and w is the unknown parameter vector.

2.2 WSINDy: Weak-Form Sparse Identification of Nonlinear Dynamics

WSINDy, introduced by Messenger and Bortz (2021), is a method for discovering the governing equations of a dynamical system directly from noisy measurement data. It builds on the original SINDy framework (Brunton et al., 2016), which poses equation discovery as a sparse regression problem: given a large candidate library of possible terms (polynomials, trigonometric functions, etc.), find the smallest subset whose weighted combination reproduces the observed dynamics. The standard SINDy approach does this by approximating the time derivative of the data pointwise and solving a sparse regression against the library. This works well in low-noise settings but breaks down when the data is noisy, since numerical differentiation amplifies the noise. WSINDy avoids this issue entirely by reformulating the problem in the weak form. Rather than differentiating the data, it multiplies the equation by a compactly supported test function, integrates by parts, and solves the resulting linear system $G \cdot w \approx b$ for the coefficient vector w. Sparsity is then enforced through sequentially thresholded least squares, which repeatedly solves the regression and zeros out any coefficients whose magnitudes fall below a chosen threshold until the active set stabilizes. The result is a small set of surviving terms that describe the underlying dynamics. Because the weak form replaces pointwise differentiation with integration against a smooth test function, WSINDy remains accurate even when the measurement data is heavily corrupted by noise.

2.3 WENDy: Weak-Form Estimation of Nonlinear Dynamics

WENDy, introduced by Bortz, Messenger, and Dukic (2023), extends the same weak-form approach to a different problem: parameter estimation. Rather than discovering which terms belong in the model, WENDy assumes the structure of the governing equation is already known and focuses on recovering its coefficients as accurately as possible. The key idea of WENDy is to address a subtle issue that arises in weak-form regression: because both sides of the linear system, the matrix G and the right-hand side b, are constructed from the same noisy data, standard least squares skews the parameter estimates. WENDy implements an Iteratively Reweighted Least Squares (IRLS) scheme as a solution. At each iteration, it uses a covariance

matrix to estimate how much the noise has had an effect on each equation in the system, rescales the regression problem using a decomposition of the covariance matrix, and re-solves. The iterative nature of the IRLS solver produces substantially more accurate parameter recovery, even when noise levels are particularly high. The performance of WENDy heavily depends on the choice of test functions, specifically, how wide their radius is. A test function that is too narrow produces a large error from the approximation of its integral, and if left too wide, leaves too few equations in the system.

2.4 The Original Codebase

The code that accompanied the research paper was developed primarily for generating the experimental results reported in the manuscript. It was organized as a flat collection of Python scripts and Jupyter notebooks within a single GitHub repository, with no formal package structure, no installation mechanism, and minimal documentation. Functions for data handling, test function construction, OLS regression, IRLS iteration, and forward simulation were interleaved or duplicated across files. Configuration was handled through long argument lists and cryptic parameter names, and there was no separation between the library's core algorithms and the experiment-specific driver code. While this organization is common in research codebases developed under time pressure, it presented significant barriers to adoption. A new user wishing to apply WENDy to their own problem would need to reverse-engineer the control flow across multiple files, understand implicit conventions (such as the meaning of integer flags), and manually adapt experiment scripts. Extending the library with new test function families, new solvers, or new problem types would require modifying existing files in ways that risked breaking the original functionality.

3 Library Design Approach

The main goal with developing pyWSciML was to transform the original research code into a library that other researchers could easily understand and eventually build onto. The main principles that guided this effort were:

1. **Separation of concerns.** Each stage of the WENDy pipeline (data storage, test function construction, OLS regression, IRLS iteration, sparsification, and forward simulation) should live in its own module with a clear, single responsibility.

2. **Inheritance over duplication.** The solvers in WENDy naturally build on one another: IRLS requires an initial OLS solve, and sparsification also relies on the OLS machinery. Rather than duplicating code or requiring the user to manually chain function calls in the right order, the library uses class inheritance so that each solver automatically has access to everything it needs from the layer below it.

3. **Readable, documented code.** The structured library includes docstrings for all public methods, descriptive parameter names where possible, and inline comments explaining the purpose of non-obvious operations. The intent is that a researcher reading the code can follow what is happening without needing to cross-reference the paper often.

4. **Clean entry points.** A user should be able to set up a problem, run a solver, and simulate the result in a small number of straightforward calls.

4 Library Structure

The refactored library is organized into five Python modules, each containing a single class. These classes are related through an inheritance hierarchy that mirrors the logical dependency between the algorithms.

4.1 WData

`WData` is the base class and serves as the data container for the entire pipeline. It stores the observed state data, the time grid, the user-specified feature library, and all of the configuration parameters that control test

function construction and solver behavior (test function type, polynomial order, SVD settings, gap, finite-difference accuracy orders, and convergence tolerances). Every other class in the library inherits from `WData`, which means that once a user passes their data and configuration into any solver, all of that information is available throughout the pipeline without needing to be passed around manually.

4.2 OLS_Solver

`OLS_Solver` inherits from `WData` and implements the baseline ordinary least squares solve. Its main method, `fit_OLS()`, handles three tasks: constructing the test function matrices (both the test functions and their derivatives), selecting the test function radius (either automatically through a corner-point heuristic or from user-specified values), and assembling and solving the weak-form linear system $G \cdot w = b$. It also supports both single-scale local and multi-scale global test function configurations, as well as optional SVD compression of the test function matrices. The matrices and solution it produces are stored as attributes and are inherited by the downstream solvers.

4.3 IRLS_Solver

`IRLS_Solver` inherits from `OLS_Solver` and implements the iteratively reweighted least squares algorithm that is the core of WENDy. Its main method, `fit_IRLS()`, first calls the inherited OLS fit to get an initial estimate, then enters the IRLS loop. At each iteration it builds a covariance matrix from a symbolic Jacobian of the library functions, estimates the noise variance from the data, computes a Cholesky factorization to precondition the system, re-solves, and checks convergence. The stopping criteria include a relative change tolerance on the parameter iterates, a maximum iteration count, and a Shapiro-Wilk normality test on the residuals. If the iterates diverge, the solver falls back to the iterate with the best p-value rather than returning a poor estimate.

4.4 SparsifyDynamicsSolver

`SparsifyDynamicsSolver` inherits from `OLS_Solver` and implements the sequential thresholding sparsification used in WSINDy. Its main method, `sparsifyDynamics()`, builds the weak-form system through the inherited OLS machinery and then applies iterative hard thresholding independently on each governing equation. At each pass, coefficients whose magnitudes fall below a user-specified threshold are zeroed out and the system is re-solved on the remaining active terms until the support stabilizes. The solver also supports Tikhonov regularization and per-coefficient masking with adaptive bounds for constrained problems.

4.5 Simulation

`Simulation` is a standalone class (it does not inherit from the solver hierarchy) that handles forward integration of the recovered model. Given a feature library, a coefficient vector from any of the solvers, an initial condition, and a time grid, its `simulate()` method integrates the reconstructed ODE system using SciPy's `solve_ivp` with the BDF method. It includes a blowup event that terminates integration early if the solution norm exceeds a threshold, and pads the output to match the expected time grid length if early termination occurs.

4.6 Class Hierarchy

The inheritance structure of the library can be summarized as follows:

```
WData
   OLS_Solver
         IRLS_Solver
         SparsifyDynamicsSolver

Simulation  (standalone)
```

This hierarchy ensures that shared functionality (data storage, test function construction, the OLS solve) is implemented once and inherited by the solvers that depend on it, rather than being duplicated.

5 Demonstration Notebook

The library includes a Jupyter notebook (`pyWSciML_script.ipynb`) that demonstrates the full pipeline on several test systems. The notebook defines three ODE systems for testing: the Lorenz system with a minimal (known-structure) library, a logistic growth model, and the Lorenz system with an overcomplete degree-2 monomial library containing 10 candidate terms per equation. For each system, the notebook generates clean trajectory data, adds controllable Gaussian noise, and then runs the relevant solvers. The Lorenz and logistic growth examples are used to demonstrate both the OLS and IRLS solvers in single-scale local and multi-scale global configurations. The overcomplete Lorenz example is used to demonstrate the sparsification solver, which must recover the few active terms from the large candidate library. In each case, the notebook computes parameter recovery error and forward simulation error, and includes plotting code for visual comparison of the recovered trajectories against the ground truth.

6 Acknowledgments

All of the algorithms implemented in this library are the original research contributions of David M. Bortz and April Tran at the University of Colorado Boulder, along with their collaborators. The underlying code was written by Tran and Bortz to accompany their paper "Weak Form Scientific Machine Learning: Test Function Construction for System Identification" (Tran & Bortz, 2025), which also builds on the earlier WENDy and WSINDy methods developed by Bortz and Messenger. The work presented here is not original research and does not introduce new algorithms or methods. My contribution was limited to reorganizing and refactoring the existing codebase into a structured, object-oriented Python library so that it can be more easily understood, used, and extended by other researchers.